

Lab 8: Trees and Tree Traversals

Algorithms and Data Structures (010153523) | Semester 1/2026

Duration: 2 hours (in-lab) | **Topic:** Binary tree, preorder/inorder/postorder, depth, height

Objectives

- Implement a binary tree using a linked-node structure with **left** and **right** children.
- Implement preorder, inorder, and postorder traversals recursively.
- Compute the depth of a node and the height of a tree recursively.
- Count nodes, count leaves, and evaluate an expression tree.

Quick Reference

Node class.

```
class Node<E> {
    E element;
    Node<E> left, right, parent;
    Node(E e, Node<E> p) { element = e; parent = p; }
}
```

Traversals.

```
void preorder(Node<E> v) { visit(v); preorder(v.left); preorder(v.right); }
void inorder(Node<E> v)  { inorder(v.left); visit(v); inorder(v.right); }
void postorder(Node<E> v) { postorder(v.left); postorder(v.right); visit(v); }
```

Height and depth.

```
int height(Node<E> v) {
    if (v == null) return -1;
    return 1 + Math.max(height(v.left), height(v.right));
}
int depth(Node<E> v) { return v.parent == null ? 0 : 1 + depth(v.parent); }
```

Quick Experiments (Try It)

Try It 1: Order of traversals

Build the tree: A at root; left child B with children D and E; right child C with right child F. Print the preorder, inorder, and postorder sequences. Verify manually.

Try It 2: Height of a skewed tree

Build a right-chain: root $\rightarrow 1 \rightarrow 2 \rightarrow \dots \rightarrow 9$. Compute the height. What is the height formula for n nodes in a right-skewed tree? Compare with a perfect binary tree of n nodes.

In-Lab Exercises (target: 2 hours)

In-Lab Exercise 1: Build and Traverse a Binary Tree (45 min)

Implement `BinaryTree<E>` with:

- Methods: `root()`, `left(Node)`, `right(Node)`, `parent(Node)`.
- `addRoot(E e)`, `addLeft(Node p, E e)`, `addRight(Node p, E e)`.
- `int size()`, `boolean isEmpty()`.
- Traversals: `preorderPrint()`, `inorderPrint()`, `postorderPrint()`.

Construct a 7-node tree and verify all three traversal outputs.

In-Lab Exercise 2: Height, Depth, and Leaf Count (30 min)

Add:

- `int height()` — height of the whole tree.
- `int depth(Node<E> v)` — depth of node `v`.
- `int countLeaves()` — nodes with no children.
- `int countNodes()` — total nodes.

Test on at least two trees (balanced vs. skewed).

In-Lab Exercise 3: Expression Tree Evaluator (25 min)

Represent $(3 + 4) \times (8 - 5)$ as a binary tree (leaves = integers, internal = operators). Write recursive `int evaluate(Node<String> v)`. Print the inorder traversal (infix expression) and the evaluated result.

In-Lab Exercise 4: Level-Order Traversal (20 min)

Implement `void levelOrder()` using a Queue: enqueue root; while non-empty, dequeue a node, visit it, enqueue non-null children. Print the output for your tree from Exercise 1.

AI Collaboration

Suggested prompts:

- “What is the difference between preorder, inorder, and postorder? Give a use case for each.”
- “Explain why inorder traversal of a BST yields sorted order.”

Verify: Ask the AI for the three traversal sequences of a specific 7-node tree. Build that tree in code and compare outputs.

Take-Home (Paper Work). Solve the following on paper without using a computer or IDE. Hand-write your answers; submit at the start of the next lab. Goal: prepare you to trace and reason about code during the written exam.

Trace & Predict 1: Traversal sequences

Write the preorder, inorder, and postorder for the tree:

```
  +
 /  \
*    7
 /  \
3    4
```

(Root +; left subtree * with children 3 and 4; right child 7.)

Write Code on Paper 1: Mirror a tree

Write recursive `void mirror(Node<E> v)` that swaps left and right children of every node. After `mirror(root)`, inorder should be the reverse of the original.

Draw on Paper 1: Tree from traversals

Reconstruct the unique binary tree with preorder A, B, D, E, C, F and inorder D, B, E, A, F, C . Draw the resulting tree and label every edge.

Submission. Save files as `lab8_ex*.java` and submit on the course site.